Assignment 3 (Mini-Project 1)
NLP with Deep Learning
Due Date: 5th April (1155 PM)
Marks = 7.5

NOTE: Do not submit Google Drive or shared document links, as they can be modified after submission. The report must be submitted as a **Word or PDF** file on the **LMS**. If other files exceed the size limit, they may be uploaded via Dropbox. However, any submission via shared links will be considered incomplete and will not be graded.

RAG-based Question-Answering System Development

1. Introduction

This is a group assignment where the group size can be of 2-3 members. In this assignment, your task is to:

- Develop a Retrieval-Augmented Generation (RAG) based question-answering system.
- The system should retrieve information from a specific domain.
- You must move beyond basic semantic search. Your system must implement Hybrid Search, Re-Ranking, and Automated Evaluation (LLM-as-a-Judge). You will also deploy a live web interface for interaction.

Core Goals:

- Implement advanced retrieval strategies (Hybrid/RRF/Re-ranking).
- Deploy a live web application on Hugging Face Spaces or any other hosting service.
- Evaluate system performance using LLM-as-a-Judge (Faithfulness & Relevancy).
- Conduct an Ablation Study to justify design choices.

Domain Corpus Requirements:

- Your corpus must contain at least 50-100 documents or 500+ chunks of text
- You may choose any domain: medical, legal, human resources, scientific papers, company policies, etc.
- Clearly document the source of your corpus and provide access (if publicly available) or include a representative sample in your report

2. Technical Stack Requirements

To ensure consistency and production readiness, all groups must adhere to the following infrastructure constraints:

Component	Requirement	Notes
Hosting	Hugging Face Spaces or any other free service	Streamlit or Gradio based interface. Must be publicly accessible.
Vector DB	Pinecone (Free Starter)	Cloud-based. Do not store indexes locally on the host.

Embeddings	Pre-computed	Generate document embeddings locally → Upsert to Pinecone.
Query Embedding	On-App	Use a lightweight model (e.g., all-MiniLM-L6-v2) directly in the Space for query encoding.
LLM Generation	HF Inference API	Use models like Mistral-7B, Llama-3-8B, or TinyAya via API
Retrieval	Hybrid + Re-ranking	Mandatory implementation of BM25 + Semantic Search with RRF or Cross-Encoder.

2A. Evaluation Protocol: LLM-as-a-Judge

Do not rely solely on human inspection. You must implement an automated evaluation pipeline using an LLM API on a fixed test set of 10-20 queries:

Faithfulness: Implement Claim Extraction & Verification.

- Extract claims from the generated answer.
- Verify each claim against the retrieved context using an LLM.
- Report the Faithfulness Score (% of claims supported).
- In your report, show extracted claims and verification results for at least 3 example queries

Relevancy: Implement Alternate Query Generation.

- Generate 3 questions from the generated answer.
- Compute cosine similarity between generated questions and the original query.
- Report the Average Relevancy Score (mean of 3 similarity scores).

2B. Ablation Study

You must include a table in your report comparing performance across variations:

- Chunking: Compare at least two strategies (e.g., Fixed vs. Recursive vs. Semantic).
- Retrieval: Compare Semantic-Only vs. Hybrid + Re-ranking.
- Metrics: Report Faithfulness and Relevancy scores for each variation.

2C. Live Web Interface

Deploy a working UI on Hugging Face Spaces or a free hosting site. The UI must allow a user to input a query and display:

- The Generated Answer
- The Retrieved Context (chunks)
- Faithfulness and Relevance scores
- (Optional) source citations

3. Report Guidelines

Organize your report into clearly defined sections. Your report should include the following components to ensure that someone else can replicate your process.

A. **Platform Details**: Specify the platform used for experimentation (e.g., local machine, Kaggle, Colab). If multiple platforms were used, clarify where each stage was executed.
B. **Data Details**: Clearly state the source of the dataset, including the size and number of documents used in the corpus.
C. **Algorithms, Models, and Retrieval Methods**: Clearly document the experimental setup and results, highlighting insights gained from multiple trials.

    - Describe the retrieval methods employed in your system. Did you use semantic search, keyword-based search, or another method? Justify your approach.
    - Specify the algorithms and large language models (LLMs) you used and explain your choices.
    - Explain your chunking strategy, including how you segmented the documents and whether different chunking approaches were tested. Discuss how chunk size and overlap affected retrieval and answer quality.
    - Explain the prompt structure used for the LLM generation and for LLM-as-a-judge.

D. **Performance Metrics**: Compare results across different models, retrieval strategies, and parameter settings, providing insights into how various choices impact performance.

    - Implement and report evaluation metrics for generated answers, specifically faithfulness and relevance. You may use automated libraries like RAGAS but if they do not work reliably, design your own prompting method to evaluate these metrics.
    - These evaluation metrics should be analyzed across different retrieval strategies, LLM choices, chunking methods, etc.
    - Moreover, analyze the latency and computational efficiency by measuring inference time, retrieval time, and overall system response time.

E. Best Model Selection: Justify your best model selection by assessing the effectiveness of using the above-mentioned performance metrics.
F. **Reproducibility**: Your report must provide enough detail to enable others to replicate your work. Include any information that is critical for reproduction, such as preprocessing steps, system configuration, or model fine-tuning techniques.

4. Urdu Low-Resource Language Bonus Track (+10%)

Groups may opt to build their RAG system for Urdu to earn a 10% bonus. This is optional but strict. Partial attempts do not qualify. To qualify for the bonus, ALL the following criteria must be met:

- Working Live Demo: The HF Spaces link must accept native Urdu script (Nastaliq) queries and return answers. (Romanized Urdu does not qualify).
- Urdu Embeddings: You must use a multilingual or Urdu-specific embedding model (e.g., paraphrase-multilingual-MiniLM-L12-v2, bge-multilingual-base). English-only models will not work for Urdu.
- Hybrid Search: You must still implement Hybrid Search (BM25 + Semantic) with RRF or Re-ranking.
- Evaluation Adaptation: Since LLM judges are English-biased, you must use a multilingual LLM (e.g., Qwen-7B, Aya-23) for evaluation.
- Challenge Report: Include a 1-page Appendix documenting Urdu-specific challenges encountered (e.g., tokenization, script handling, data scarcity, judge bias) and how you mitigated them.

5. Additional Instructions

- **Figures and Tables**: Ensure that all figures and tables are properly numbered and cited in the text. Avoid vague references like "the figure below"; instead, use precise citations such as "Table 1 shows…" or "As shown in Figure 7…".
- **Code Submission**: Either include the working code in your submission or provide a link to a repository (e.g., GitHub) where the code can be accessed. The date of upload must be before the deadline.
- **References**: If you have used external resources, such as blogs or GitHub repositories, ensure they are appropriately cited. Include a reference section before the appendix to acknowledge all sources and avoid any potential issues of plagiarism. Proper citation is a key part of your academic and professional training.
- **Submission File Name**: The file name should be as per the group members name and don't name it Assignment1 or Project1. So if there are two group members, Aamna and Zaid, then name it Aamna_Zaid.docx.